Queue in Java
Complete Concept

Java DSA Study Notes

Full Queue concept notes • implementations • Java collections • BFS • LeetCode

Contents preserved from the supplied Queue lesson

Java code and diagrams are preserved in monospace with readable indentation and line breaks.

Table of Contents

1. 2. Queue vs Stack
2. 3. Important Queue Terminology
3. 4. Basic Queue Operations
4. 5. Enqueue
5. 6. Dequeue
6. 7. Peek
7. 8. `isEmpty()`
8. 9. `size()`
9. 10. Complete Queue Operation Example
10. 11. Queue Using an Array
11. 12. What does `front` mean?
12. 13. What does `rear` mean?
13. 14. Enqueue implementation
14. 15. Another Enqueue
15. 16. Dequeue implementation
16. 17. Peek implementation
17. 18. `isEmpty()` implementation
18. 19. `size()` implementation
19. 20. Complete Array Queue
20. 21. Problem with Normal Array Queue
21. 22. Circular Queue
22. 23. The Important Circular Formula
23. 24. Circular Queue Implementation
24. 25. Why do we maintain `size`?
25. 26. Queue Using Linked List
26. 27. Enqueue in Linked List
27. 28. Dequeue in Linked List
28. 29. Special Case: Removing the Last Element
29. 30. Complete Linked List Queue
30. 31. Why Linked List Queue is useful
31. 32. Java's `Queue` Interface
32. 33. Queue with `LinkedList`
33. 34. Queue with `ArrayDeque`
34. 35. Why `ArrayDeque`?
35. 36. Queue Methods — Complete Important List
36. 37. `add()` vs `offer()`
37. 38. `remove()` vs `poll()`
38. 39. `element()` vs `peek()`
39. 40. Queue Method Table
40. 41. `PriorityQueue`
41. 42. Normal Queue vs PriorityQueue

42. 43. Max PriorityQueue
43. 44. Deque
44. 45. Queue vs Deque
45. 46. `ArrayDeque` as Deque
46. 47. Queue vs Stack vs Deque
47. 48. Time Complexity
48. 49. Important Queue Edge Cases
49. 50. Where are Queues actually used?
50. 51. Important Queue LeetCode Problems
51. 52. What you should remember

Queue in Java — Complete Concept

1. What is a Queue?

A Queue is a linear data structure that follows:

> **FIFO — First In, First Out**

Think about people standing in a line:

```
A → B → C → D
↑           ↑
Front       Rear
```

A entered first, so A leaves first.

If we add E:

```
A → B → C → D → E
↑               ↑
Front           Rear
```

If we remove one:

```
B → C → D → E
↑           ↑
Front       Rear
```

So the fundamental rule is:

```
INSERT  → REAR
REMOVE  → FRONT
```

2. Queue vs Stack

You should know this difference perfectly.

Stack

```
LIFO
Last In → First Out
```

```
       30 ← remove
       20
       10
```

Queue

```
FIFO
First In → First Out
```

```
10 → 20 → 30
↑           ↑
remove      insert
```

StackQueue	
LIFO	FIFO
Insert at top	Insert at rear
Remove from top	Remove from front
`push()`	`offer()`
`pop()`	`poll()`
`peek()`	`peek()`

3. Important Queue Terminology

There are two ends.

Front

The front is where we remove elements.

```
FRONT
  ↓
[10] [20] [30]
```

Rear

The rear is where we insert elements.

```
             REAR
              ↓
[10] [20] [30]
```

Therefore:

```
Front → removal
Rear  → insertion
```

4. Basic Queue Operations

A Queue has five fundamental operations:

```
1. Enqueue
2. Dequeue
3. Peek
4. isEmpty
5. size
```

Let's understand each.

5. Enqueue

Enqueue = insert an element.

Suppose:

```
[10] [20] [30]
 ↑           ↑
Front       Rear
```

Perform:

```
enqueue(40)
```

Result:

```
[10] [20] [30] [40]
 ↑                  ↑
Front              Rear
```

So:

```
ENQUEUE → INSERT AT REAR
```

6. Dequeue

Dequeue = remove an element.

Suppose:

```
[10] [20] [30]
 ↑
Front
```

Perform:

```
dequeue()
```

`10` is removed.

```
[20] [30]
 ↑       ↑
Front   Rear
```

So:

```
DEQUEUE → REMOVE FROM FRONT
```

7. Peek

`peek()` means:

> **Look at the front element without removing it.**

Suppose:

```
[10] [20] [30]
 ↑
Front
```

```
peek();
```

returns:

```
10
```

But the Queue stays:

```
[10] [20] [30]
```

Therefore:

```
poll() → remove
peek() → just look
```

8. `isEmpty()`

Checks whether the Queue has zero elements.

```
[]
```

```
isEmpty()
```

returns:

```
true
```

But:

```
[10] [20]
```

returns:

```
false
```

9. `size()`

Returns the number of elements.

```
[10] [20] [30]
```

Therefore:

```
size()
```

returns:

```
3
```

10. Complete Queue Operation Example

Start:

```
[]
```

enqueue(10)

```
[10]
```

enqueue(20)

```
[10] [20]
```

enqueue(30)

```
[10] [20] [30]
```

peek()

Returns:

```
10
```

Queue remains:

```
[10] [20] [30]
```

dequeue()

Returns:

```
10
```

Queue:

```
[20] [30]
```

enqueue(40)

```
[20] [30] [40]
```

dequeue()

Returns:

```
20
```

Queue:

```
[30] [40]
```

That's FIFO.

11. Queue Using an Array

Now let's implement Queue ourselves.

Suppose:

```
int[] arr = new int[5];
```

Initially:

```
[ ] [ ] [ ] [ ] [ ]
```

We need two variables:

```
int front;
int rear;
```

Initially:

```
front = 0;
rear = 0;
```

Think of them as positions.

12. What does `front` mean?

`front` tells us where the current front element is.

Example:

```
[10] [20] [30]
 ↑
front
```

Here:

```
front = 0
```

After removing `10`:

```
[10] [20] [30]
      ↑
    front
```

Now:

```
front = 1
```

We don't necessarily physically erase `10`.

We simply move `front`.

13. What does `rear` mean?

In the implementation we're using, `rear` points to the next empty position.

Initially:

```
[ ] [ ] [ ] [ ] [ ]
 ↑
front/rear
```

```
front = 0
rear = 0
```

After inserting `10`:

```
[10] [ ] [ ] [ ] [ ]
 ↑       ↑
front   rear
```

Now:

```
front = 0
rear = 1
```

Why `rear = 1`?

Because index `1` is the next position where we can insert.

14. Enqueue implementation

```
void enqueue(int value) {
    arr[rear] = value;
    rear++;
}
```

Let's execute:

```
enqueue(10);
```

Initially:

```
rear = 0
```

Therefore:

```
arr[0] = 10;
```

Then:

```
rear++;
```

Now:

```
rear = 1
```

15. Another Enqueue

```
enqueue(20);
```

Current:

```
rear = 1
```

Therefore:

```
arr[1] = 20;
```

Then:

```
rear = 2
```

Array:

```
[10] [20] [ ] [ ] [ ]
 ↑           ↑
front       rear
```

16. Dequeue implementation

```
int dequeue() {
    int value = arr[front];
    front++;
    return value;
}
```

Suppose:

```
[10] [20] [30]
 ↑
front
```

`front = 0`.

Execute:

```
int value = arr[front];
```

Therefore:

```
value = arr[0];
```

So:

```
value = 10
```

Then:

```
front++;
```

Now:

```
front = 1
```

Finally:

```
return value;
```

returns:

```
10
```

17. Peek implementation

Very simple:

```
int peek() {
    return arr[front];
}
```

It doesn't modify `front`.

Therefore it doesn't remove anything.

18. `isEmpty()` implementation

For this simple implementation:

```
boolean isEmpty() {
    return front == rear;
}
```

Why?

Suppose:

```
front = 0
rear = 0
```

There are no elements.

```
front == rear
```

is true.

After inserting three:

```
front = 0
rear = 3
```

So:

```
front != rear
```

Queue isn't empty.

19. `size()` implementation

```
int size() {
    return rear - front;
}
```

Example:

```
front = 2
rear = 5
```

Therefore:

```
size = 5 - 2
     = 3
```

20. Complete Array Queue

```
class Queue {

    int[] arr;
    int front;
    int rear;

    Queue(int size) {
        arr = new int[size];
        front = 0;
        rear = 0;
    }

    void enqueue(int value) {
        arr[rear] = value;
        rear++;
    }

    int dequeue() {
        int value = arr[front];
        front++;
        return value;
    }

    int peek() {
        return arr[front];
    }

    boolean isEmpty() {
        return front == rear;
    }

    int size() {
        return rear - front;
    }
}
```

This teaches the basic mechanics.

But there is a problem.

21. Problem with Normal Array Queue

Suppose array size is 5.

We insert:

```
10 20 30 40 50
```

Array:

```
[10] [20] [30] [40] [50]
 ↑                    ↑
front                rear
```

Now remove:

```
10
20
30
```

Logically:

```
[40] [50]
```

But physically:

```
[10] [20] [30] [40] [50]
              ↑       ↑
           front    rear
```

The first three positions are free.

But `rear` has reached the end.

If we try:

```
enqueue(60);
```

we try:

```
arr[5] = 60;
```

But the last valid index is `4`.

So we get an array bounds error.

This is why we need:

Circular Queue

22. Circular Queue

A Circular Queue treats the array as a circle.

Normal array:

```
0 → 1 → 2 → 3 → 4
```

Circular:

```
0 → 1 → 2 → 3 → 4
↑                ↓
└────────────────┘
```

After index `4`, we return to index `0`.

23. The Important Circular Formula

```
index = (index + 1) % size;
```

Suppose:

```
size = 5
index = 3
```

Then:

```
(3 + 1) % 5
= 4
```

Next:

```
(4 + 1) % 5
= 0
```

So:

```
3 → 4 → 0 → 1 → 2 → 3
```

This is the key idea behind circular arrays.

24. Circular Queue Implementation

A simple implementation:

```
class CircularQueue {

    int[] arr;
    int front;
    int rear;
    int size;

    CircularQueue(int capacity) {
        arr = new int[capacity];
        front = 0;
        rear = 0;
        size = 0;
    }

    void enqueue(int value) {

        if (size == arr.length) {
            System.out.println("Queue is full");
            return;
        }

        arr[rear] = value;

        rear = (rear + 1) % arr.length;

        size++;
    }

    int dequeue() {

        if (size == 0) {
            System.out.println("Queue is empty");
            return -1;
        }

        int value = arr[front];

        front = (front + 1) % arr.length;

        size--;

        return value;
    }

    int peek() {

        if (size == 0) {
            return -1;
        }

        return arr[front];
    }

    boolean isEmpty() {
        return size == 0;
    }

    boolean isFull() {
        return size == arr.length;
    }
}
```

The most important difference from the normal Queue is:

```
rear = (rear + 1) % arr.length;
```

and:

```
front = (front + 1) % arr.length;
```

25. Why do we maintain `size`?

In a Circular Queue, `front` and `rear` can sometimes become equal in two different situations:

Queue empty

```
front == rear
```

Queue completely full

After wrapping around:

```
front == rear
```

So just comparing:

```
front == rear
```

isn't enough.

We keep:

```
size
```

to distinguish them.

```
size == 0          → empty
size == capacity   → full
```

26. Queue Using Linked List

Now let's implement Queue using the Linked List concept you've already learned.

Node:

```
class Node {

    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Queue needs:

```
Node front;
Node rear;
```

Visual:

```
front
  ↓
[10] → [20] → [30] → null
                      ↑
                    rear
```

27. Enqueue in Linked List

Suppose:

```
front
  ↓
[10] → [20] → null
               ↑
             rear
```

We want:

```
enqueue(30)
```

Create:

```
Node newNode = new Node(30);
```

Connect:

```
rear.next = newNode;
```

Now:

```
[10] → [20] → [30] → null
```

Move rear:

```
rear = newNode;
```

Now:

```
front
  ↓
[10] → [20] → [30] → null
                      ↑
                    rear
```

28. Dequeue in Linked List

Suppose:

```
front
  ↓
[10] → [20] → [30] → null
                      ↑
                    rear
```

We remove `10`.

Simply:

```
front = front.next;
```

Now:

```
front
  ↓
[20] → [30] → null
               ↑
              rear
```

Again, we don't need to physically delete the old node manually.

29. Special Case: Removing the Last Element

This is important.

Suppose:

```
front
  ↓
[10]
 ↑
rear
```

There is only one element.

After:

```
front = front.next;
```

we get:

```
front = null
```

But `rear` still points to the old node.

That's wrong.

So we need:

```
if (front == null) {
    rear = null;
}
```

Therefore:

```
int dequeue() {

    if (front == null) {
        return -1;
    }

    int value = front.data;

    front = front.next;

    if (front == null) {
        rear = null;
    }

    return value;
}
```

This is an important Linked List Queue edge case.

30. Complete Linked List Queue

```
class Queue {

    class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
        }
    }

    Node front;
    Node rear;

    void enqueue(int value) {

        Node newNode = new Node(value);

        if (rear == null) {
            front = rear = newNode;
            return;
        }

        rear.next = newNode;
        rear = newNode;
    }

    int dequeue() {

        if (front == null) {
            return -1;
        }

        int value = front.data;

        front = front.next;

        if (front == null) {
            rear = null;
        }

        return value;
    }

    int peek() {

        if (front == null) {
            return -1;
        }

        return front.data;
    }

    boolean isEmpty() {
        return front == null;
    }
}
```

This is the fundamental Linked List Queue implementation.

31. Why Linked List Queue is useful

With a properly implemented Linked List Queue:

```
enqueue → O(1)
dequeue → O(1)
peek    → O(1)
```

Because:

```
enqueue → directly use rear
dequeue → directly use front
```

No shifting is required.

32. Java's Queue Interface

Now we move from implementing Queue ourselves to Java's built-in Queue.

Java provides:

```
Queue<E>
```

It is an interface.

So:

```
Queue<Integer> queue = new Queue<>();
```

is invalid.

Instead, we use an implementation:

```
Queue<Integer> queue = new LinkedList<>();
```

or:

```
Queue<Integer> queue = new ArrayDeque<>();
```

33. Queue with LinkedList

```
Queue<Integer> queue = new LinkedList<>();
```

Then:

```
queue.offer(10);
queue.offer(20);
queue.offer(30);
```

Queue:

```
[10, 20, 30]
```

Remove:

```
queue.poll();
```

returns:

```
10
```

Queue:

```
[20, 30]
```

34. Queue with `ArrayDeque`

This is another very important implementation:

```
Queue<Integer> queue = new ArrayDeque<>();
```

Then:

```
queue.offer(10);
queue.offer(20);
queue.offer(30);
```

and:

```
queue.poll();
queue.peek();
```

This is a very common Java DSA pattern.

For example:

```
Queue<Integer> queue = new ArrayDeque<>();

queue.offer(10);
queue.offer(20);
queue.offer(30);

System.out.println(queue.poll());
System.out.println(queue.peek());
```

Output:

```
10
20
```

35. Why `ArrayDeque`?

You already learned `ArrayDeque` for Stack.

It can be used for both:

Stack

```
Deque<Integer> stack = new ArrayDeque<>();

stack.push(10);
stack.push(20);

stack.pop();
```

Queue

```
Queue<Integer> queue = new ArrayDeque<>();

queue.offer(10);
queue.offer(20);

queue.poll();
```

So:

```
ArrayDeque
   │
   ├── Stack behavior
   │
   └── Queue behavior
```

36. Queue Methods — Complete Important List

Now let's organize the Java methods.

Insertion

```
add()
offer()
```

Both insert at the rear.

Removal

```
remove()
poll()
```

Both remove from the front.

Examination

```
element()
peek()
```

Both examine the front.

The difference is their behavior when the Queue is empty.

37. `add()` vs `offer()`

```
queue.add(10);
```

and:

```
queue.offer(10);
```

Both normally insert.

Difference:

```
add()
→ exception if insertion cannot happen

offer()
→ false if insertion cannot happen
```

For normal DSA code, you'll often see:

```
queue.offer(x);
```

38. `remove()` vs `poll()`

Both remove front.

```
queue.remove();
```

vs:

```
queue.poll();
```

If Queue is empty:

```
remove() → NoSuchElementException
poll()   → null
```

So `poll()` is safer when you want empty-queue handling without an exception.

39. `element()` vs `peek()`

Both inspect front.

If Queue is:

```
[10] [20] [30]
```

both return:

```
10
```

If empty:

```
element() → exception
peek()    → null
```

Therefore, in most DSA code:

```
queue.peek();
```

is convenient.

40. Queue Method Table

Memorize this:

PurposeException versionSafe/special-return version		
Insert	`add()`	`offer()`
Remove	`remove()`	`poll()`
Examine	`element()`	`peek()`

The three methods you'll use most:

```
offer()
poll()
peek()
```

Think:

```
offer → put
poll  → take
peek  → look
```

41. `PriorityQueue`

Now we reach another major Queue concept.

A normal Queue is:

```
FIFO
```

But `PriorityQueue` is based on priority, not ordinary FIFO order.

Example:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.offer(30);
pq.offer(10);
pq.offer(20);
```

Now:

```
pq.poll();
```

returns:

```
10
```

Next:

```
20
```

Next:

```
30
```

By default, Java's `PriorityQueue<Integer>` is a min-heap.

So the smallest element has highest priority.

42. Normal Queue vs PriorityQueue

Normal:

```
offer(30)
offer(10)
offer(20)
```

Removal order:

```
30 → 10 → 20
```

because it's FIFO.

PriorityQueue:

```
offer(30)
offer(10)
offer(20)
```

Removal order:

```
10 → 20 → 30
```

because priority determines removal.

43. Max PriorityQueue

If you want the largest element first:

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>(Collections.reverseOrder());
```

Then:

```
pq.offer(10);
pq.offer(30);
pq.offer(20);
```

`poll()` gives:

```
30
```

then:

```
20
```

then:

```
10
```

44. Deque

After Queue, you should know Deque.

Deque means:

> **Double Ended Queue**

You can insert and remove from both ends.

```
        FRONT
          ↓
[10] [20] [30] [40]
                  ↑
                REAR
```

Operations can happen on both sides:

```
addFirst()
addLast()

removeFirst()
removeLast()
```

This is why `ArrayDeque` is so powerful.

45. Queue vs Deque

Queue

Normally:

```
insert → rear
remove → front
```

Deque

```
insert front
insert rear

remove front
remove rear
```

Visual:

```
          FRONT                 REAR
            ↓                     ↓
          [10] [20] [30] [40]

addFirst(5)

          [5] [10] [20] [30] [40]

addLast(50)

          [5] [10] [20] [30] [40] [50]

removeFirst()

          [10] [20] [30] [40] [50]

removeLast()

          [10] [20] [30] [40]
```

46. ArrayDeque as Deque

```
Deque<Integer> deque = new ArrayDeque<>();
```

Insert:

```
deque.addFirst(10);
deque.addLast(20);
```

Result:

```
[10, 20]
```

Remove:

```
deque.removeFirst();
```

removes:

```
10
```

And:

```
deque.removeLast();
```

removes:

```
20
```

You already learned these methods when we studied `ArrayDeque`.

47. Queue vs Stack vs Deque

This is an important final comparison.

Stack

```
LIFO

        ↓ insert
       [30]
       [20]
       [10]
        ↑ remove
```

Queue

```
FIFO

remove ← [10] [20] [30] ← insert
```

Deque

```
        insert/remove
              ↓
        [10] [20] [30]
              ↑
        insert/remove
```

Deque can behave like both Stack and Queue.

48. Time Complexity

For a properly implemented Queue:

OperationArrayDequeLinked List Queue		
Enqueue	O(1)*	O(1)
Dequeue	O(1)*	O(1)
Peek	O(1)	O(1)

OperationArrayDequeLinked List Queue		
isEmpty	O(1)	O(1)
Size	O(1)	O(1)

`ArrayDeque` operations are amortized O(1); occasional resizing can make an individual insertion more expensive.

For a PriorityQueue:

OperationComplexity	
`offer()`	O(log n)
`poll()`	O(log n)
`peek()`	O(1)

49. Important Queue Edge Cases

You should always think about:

Empty Queue

```
[]
```

What happens when:

```
poll()
peek()
```

?

For `ArrayDeque`:

```
poll() → null
peek() → null
```

One element

```
[10]
```

After:

```
poll()
```

Queue becomes:

```
[]
```

This is especially important in your own Linked List implementation because:

```
front = null;
rear = null;
```

must both eventually become `null`.

Full Circular Queue

If:

```
size == capacity
```

then:

```
Queue is full
```

50. Where are Queues actually used?

Queues are extremely important in DSA.

1. BFS

Breadth-First Search uses a Queue.

For a tree:

```
      1
     / \
    2   3
   / \
  4   5
```

BFS visits:

```
1 → 2 → 3 → 4 → 5
```

A Queue helps maintain this order.

2. Level Order Traversal

Binary tree:

```
      1
     / \
    2   3
   / \
  4   5
```

Level order:

```
1
2 3
4 5
```

Queue is the standard approach.

3. Graph BFS

Queue is fundamental to:

```
BFS
Shortest path in unweighted graph
Connected components
Grid traversal
```

4. Scheduling

Queues model:

```
CPU scheduling
Task processing
Print queues
Request processing
```

5. Producer-Consumer systems

One part produces data:

```
Producer → Queue → Consumer
```

The consumer processes items in order.

51. Important Queue LeetCode Problems

Once the concepts are comfortable, these are useful:

Basic Queue

933 — Number of Recent Calls

Good for understanding Queue.

BFS

102 — Binary Tree Level Order Traversal

Very important.

Grid BFS

994 — Rotting Oranges

Excellent Queue/BFS problem.

Graph BFS

200 — Number of Islands

Can be solved with BFS using a Queue.

Shortest Path

1091 — Shortest Path in Binary Matrix

Queue + BFS.

Design

622 — Design Circular Queue

Excellent for understanding the circular queue implementation we just discussed.

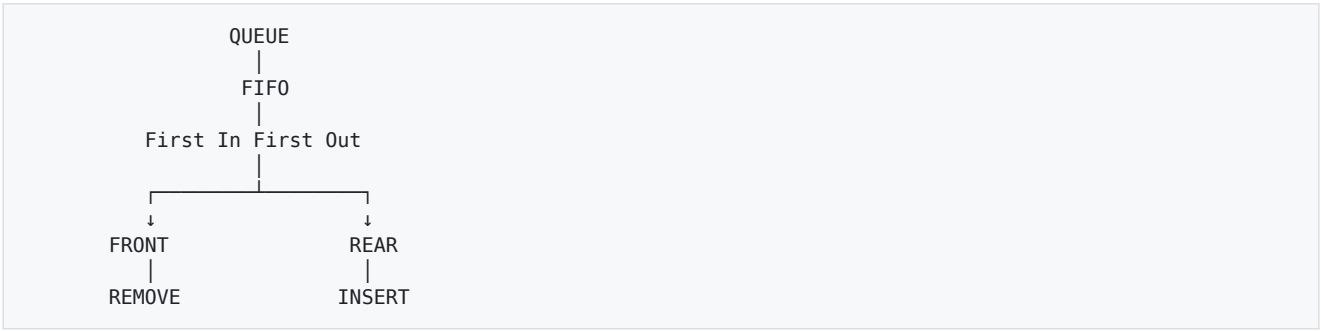
Priority Queue

215 — Kth Largest Element in an Array

Very important PriorityQueue application.

52. What you should remember

If you forget everything else, remember this:



Basic operations:

```
enqueue → insert
dequeue → remove
peek    → see front
```

Java:

```
Queue<Integer> queue = new ArrayDeque<>();

queue.offer(10);
queue.offer(20);
queue.offer(30);

queue.poll();      // 10
queue.peek();      // 20
queue.isEmpty();   // false
queue.size();      // 2
```

Java method relationships:

```
add()     ↔ offer()
remove()  ↔ poll()
element() ↔ peek()
```

Advanced Queue structures:

```
Queue
 │
 ├── Array Queue
 │
 ├── Circular Queue
 │
 ├── Linked List Queue
 │
 ├── ArrayDeque
 │
 ├── PriorityQueue
 │
 └── Deque
```

And the most important DSA connection:

```
Queue
  ↓
BFS
  ↓
Trees
  ↓
Graphs
  ↓
Shortest Paths
  ↓
Grid Problems
```